Graph Analysis via Graph Rewriting: Sparse Linear Algebra Based Graph Analysis in Interaction Nets

Dmitry S. Kulyabov[1,2,*,†], Ilaria Tiddi[3,†] and Manfred Jeusfeld[4,†]

[1]Peoples' Friendship University of Russia (RUDN University), 6 Miklukho-Maklaya St, Moscow, 117198, Russian Federation
[2]Joint Institute for Nuclear Research, 6 Joliot-Curie, Dubna, Moscow region, 141980, Russian Federation
[3]Vrije Universiteit Amsterdam, De Boelelaan 1105, 1081 HV Amsterdam, The Netherlands
[4]University of Skövde, Högskolevägen 1, 541 28 Skövde, Sweden

Abstract

A clear and well-documented LaTeX document is presented as an article formatted for publication by CEUR-WS in a conference proceedings. Based on the "ceurart" document class, this article presents and explains many of the common variations, as well as many of the formatting elements an author may use in the preparation of the documentation of their work.

Keywords

Interaction Nets, graph rewriting, graph analysis algorithms, GraphBLAS, sparse linear algebra, quad-tree

1. Introduction

Graph analysis has become fundamental to modern data science, powering applications ranging from recommendation systems in social networks [?] and fraud detection in financial transactions [?] to protein interaction analysis in bioinformatics [?] and static code analysis [?]. As graph sizes grow to billions of vertices and edges, the need for efficient, scalable implementations becomes critical. However, graph analysis algorithms are difficult to parallelize due to their irregular memory access patterns, poor data locality, and load imbalance across computational units [?].

The GraphBLAS standard [?] addresses these challenges by providing a foundation for constructing high-performance parallel graph analysis algorithms. By expressing graph algorithms in the language of linear algebra—using sparse vectors and matrices as core data structures, together with a rich set of highly parameterizable operations (matrix multiplication, element-wise operations, etc.)—GraphBLAS offers a high-level, mathematically rigorous framework for algorithm construction. Moreover, these linear algebraic primitives can be optimized for diverse hardware architectures, providing a path to high-performance graph analysis across platforms. This approach has proven successful, with implementations like SuiteSparse:GraphBLAS [?] delivering high performance on shared-memory systems, and LAGraph [?] providing a collection of graph analysis algorithms built on these primitives. However, designing parallel functions over sparse data structures remains a nontrivial engineering challenge, particularly for accelerators. Beyond CPU implementations, GraphBLAS has been partially ported to such devices: GraphBLAST [?] and Spla [?] target GPGPUs, while GraphLily [?] targets FPGAs.

Despite its successes, GraphBLAS and its implementations face certain challenges. The algebraic structures central to GraphBLAS—such as semirings and monoids—align more naturally with functional programming paradigms than with imperative ones. Generic data structures, such as matrices and vectors, and functions that required to provide a flexible core for matrix processing, together

Woodstock'22: Symposium on the irreproducible science, June 07–11, 2022, Woodstock, NY
*Corresponding author.
†These authors contributed equally.
kulyabov-ds@rudn.ru (D. S. Kulyabov); i.tiddi@vu.nl (I. Tiddi); Manfred.Jeusfeld@acm.org (M. Jeusfeld)
https://yamadharma.github.io/ (D. S. Kulyabov); https://kmitd.github.io/ilaria/ (I. Tiddi); http://conceptbase.sourceforge.net/mjf/ (M. Jeusfeld)
0000-0002-0877-7063 (D. S. Kulyabov); 0000-0001-7116-9338 (I. Tiddi); 0000-0002-9421-8566 (M. Jeusfeld)
© 2022 Copyright for this paper by its authors. Use permitted under Creative Commons License Attribution 4.0 International (CC BY 4.0).

with user-defined types and operations (which are themselves functions), complicate API design and usage, particularly with respect to type safety. This insight suggests that functional programming languages could simplify the design of GraphBLAS-like APIs, offering cleaner abstractions and more composable primitives. However, general-purpose functional languages are often not fast enough for high-performance numerical computing. Research into functional languages and frameworks for parallel computations (including GPUs)—such as Futhark [**?**], AnyDSL [**?**], LIFT [**?**], and Accelerate [**?**]—while demonstrating promising results, has largely focused on dense computations, leaving sparse irregular workloads less explored.

At the same time, interaction nets [1]—a graph-rewriting model introduced by Yves Lafont—offer a computational model that is inherently well-suited for irregular parallelism. In interaction nets, computation proceeds through local rewriting rules applied to pairs of connected nodes (so-called active pairs). Crucially, multiple non-interfering active pairs can be reduced simultaneously, making interaction nets a promising foundation for parallel irregular computations, including graph analysis. Furthermore, interaction nets are relatively close to functional programming languages, making it easy to transfer design approaches and ideas from the functional world.

In this paper, we present the design and implementation of a GraphBLAS-like linear algebra foundation in two different settings: F# (a mature functional-first language, chosen for ease of prototyping and testing) and Inpla (a language and runtime for interaction nets). We employ quadtrees to represent sparse matrices and vectors, enabling natural divide-and-conquer parallelism through recursive subdivision. On this foundation, we build three canonical graph algorithms—breadth-first search (BFS), single-source shortest paths (SSSP), and triangle counting—using the algebraic primitives provided by our API.

Our contributions are as follows.

1. !!! A design for a GraphBLAS-like API in functional settings, demonstrating how to utilize types for natural graph encoding, together with a minimal set of operations (e.g., `map`, `map2`, `spmv`, `spgemm`).
2. A minimal subset of this API implemented in both F# and Inpla, enabling a direct comparison between a conventional functional language and an interaction net-based approach.
3. An empirical evaluation of BFS, SSSP, and triangle counting using our implementations, analyzing correctness, performance, scalability, and comparison with LAGraph.

The paper is structured as follows. Section 2 provides background on GraphBLAS, quadtree matrix representations, interaction nets, and the Inpla language. Section 3 details our API design and its implementation in both F# and Inpla. Section 4 describes the graph algorithms built on this API. Section 5 presents our experimental setup and evaluation results. Section 6 discusses related work, and Section 7 concludes with directions for future research.

2. Background

2.1. GraphBLAS

Basic ideas.

Problems (implicit zeroes, etc).

SuiteSparse:GraphBLAS as a reference implementation

LAGraph as a collection of graph analysis algorithms (based on SuiteSparse:GraphBLAS).

2.2. Tree-based Representation for Sparse Matrices and Vectors

Explain the quadtree structure. Show how a matrix is recursively subdivided into quadrants (NW, NE, SW, SE) until a uniform or empty quadrant is reached. Contrast this with CSR/CSC formats used in libraries like SuiteSparse. Discuss its advantages (natural divide-and-conquer, adaptive to sparsity patterns) and potential disadvantages (overhead for very regular matrices).

2.3. Interaction Nets

Introduce Interaction Nets as a graph-rewriting model. Explain the core concepts: agents (nodes with a principal port and auxiliary ports), active pairs (two agents connected by their principal ports), and interaction rules (rewriting rules that define how active pairs are reduced).

Emphasize the key properties: one-step confluence (the order of reduction doesn't matter) and inherent parallelism (active pairs can be reduced simultaneously).

2.4. Inpla

Introduce Inpla: runtime (VM) and language. Briefly describe its syntax for defining agents and rules, and how it allows to program these graph-rewriting systems. This section sets the stage for understanding our Inpla-based implementation.

3. GraphBLAS-like API in Functional style

3.1. Design and Architecture

Architecture, Structure of modules

API: matrices, vectors, functions (map, map2, mxm, mxv, !!!)

3.2. Quad-trees

Quad-trees.

3.3. Interaction Nets

Agents

4. Implemented Graph Analysis Algorithms

implemented algorithms: BFS, SSSP, Triangles counting (name?).

5. Evaluation

5.1. Experimental Setup

Hardware. All experiments were conducted on a system with an AMD Ryzen 9 7900X 12-core processor (24 threads with hyper-threading enabled, base clock 4.7 GHz) and 128 GiB of DDR5 RAM.

Software. We compare four implementations:

- **F#**: Our quadtree-based implementation, compiled with F# !!! and the .NET !!! runtime. This serves as a baseline representing a conventional functional language approach.
- **Inpla**: Our interaction-net-based implementation, executed on the Inpla runtime system (version !!!).
- **LAGraph**: A collection of graph algorithms built on SuiteSparse:GraphBLAS (version !!!), providing state-of-the-art, linear-algebra-based reference implementations of BFS, SSSP, and triangle counting.
- **NetworkX**: A popular Python library for graph analysis (version !!!), included as a baseline representing ease of use over optimization.

Datasets. We evaluate our implementations on several real-world and synthetic graphs from the SuiteSparse Matrix Collection [**?**]. The graphs are chosen to represent a variety of sizes, sparsity patterns, and application domains. Table 1 summarizes their key properties: number of vertices ($|V|$), number of edges ($|E|$), and degree statistics (average, maximum, and median).

Table 1
Dataset properties

Graph	$\|V\|$	$\|E\|$	Avg. degree	Max degree	Median degree
roadNet-CA	!!!	!!!	!!!	!!!	!!!
cit-Patents	!!!	!!!	!!!	!!!	!!!
wikipedia-2007	!!!	!!!	!!!	!!!	!!!
…	…	…	…	…	…

Research Questions. Our evaluation addresses the following research questions:

RQ1 (Correctness): Do the F# and Inpla implementations produce correct results for BFS and triangle counting on all datasets, when compared against reference outputs from NetworkX and LAGraph?

RQ2 (Performance – Inpla vs. State-of-the-Art): How does the execution time of Inpla compare to LAGraph on BFS, SSSP, and triangle counting? We also include F# and NetworkX as additional baselines representing different points in the performance–productivity trade-off.

RQ3 (Parallel Scaling): How well do the Inpla and LAGraph implementations scale with the number of available CPU cores? We measure speedup from 1 to all available cores for BFS, SSSP, and triangle counting, with hyper-threading both enabled and disabled.

RQ4 (Impact of Matrix Reordering): Matrix reordering techniques (e.g., reverse Cuthill–McKee) can improve locality and cache performance. How does applying such reordering affect the performance of Inpla, F#, and LAGraph implementations?

5.2. Evaluation Results

Graphics, tables.
Reordering: figures with sparsity patterns
Analysis.

6. Related Work

Interaction nets
Sparse linear algebra and graphs
Functional languages for parallel computing

7. Conclusion and Future Work

Future work.
Improvements of VM.
Other algorithms on graphs.
COO format (sorting, qsort).

Acknowledgments

Thanks to the developers of ACM consolidated LaTeX styles https://github.com/borisveytsman/acmart and to the developers of Elsevier updated LaTeX templates https://www.ctan.org/tex-archive/macros/latex/contrib/els-cas-templates.

Declaration on Generative AI

During the preparation of this work, the author(s) used X-GPT-4 and Gramby in order to: Grammar and spelling check. Further, the author(s) used X-AI-IMG for figures 3 and 4 in order to: Generate images. After using these tool(s)/service(s), the author(s) reviewed and edited the content as needed and take(s) full responsibility for the publication's content.

References

[1] Y. Lafont, Interaction nets, in: Proceedings of the 17th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, POPL '90, Association for Computing Machinery, New York, NY, USA, 1989, p. 95–108. URL: https://doi.org/10.1145/96709.96718. doi:10.1145/96709.96718.